

S4.01.Cristina Fornaguera

Tasca S4.01. BigQuery Avançat & Analytics Engineering

Nivell 1: Entorn i Ingesta Híbrida (Code-First)

Escenari:

El quadre de comandament financer tarda massa a carregar i el cost és inacceptablement alt. S'ha identificat que el problema crític són els informes que creuen vendes amb regions geogràfiques.

Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic)

El Country Manager d'Alemanya necessita revisar urgentment les transaccions del dia 12 de març de 2022.

✖ Tasca:

- » 1. Escriu la consulta que uneix (JOIN) transaccions i companyies.
- » 2. Filtra els resultats per la data indicada i el país "Germany".
- » 3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).

👁 Observació: Fixa't que BigQuery llegeix gairebé tota la taula tot i demanar només un dia (Full Table Scan).

- Creamos la consulta sobre las tablas limpias de la capa Silver y nos fijamos en el Dry Run:

SELECT

```
t.transaction_id,  
t.timestamp,  
t.amount,  
t.declined,  
c.company_name,  
c.country
```

FROM

```
`sprint3_silver.transactions_clean` AS t
```

JOIN

```
`sprint3_silver.companies_clean` AS c
```

```
ON t.business_id = c.company_id
WHERE
DATE(t.timestamp) = '2022-03-12'
AND c.country = 'Germany';
```

The screenshot shows the Google Cloud BigQuery console interface. On the left, the 'sprint3-analytics-cristina-f-t' project is selected, showing a list of datasets including 'sprint3_silver' with tables like 'companies_clean', 'credit_cards_clean', 'products_clean', 'transactions_clean', and 'users_combined'. The main editor displays a query titled 'Untitled query' with the following SQL code:

```
1 SELECT
2   t.transaction_id,
3   t.timestamp,
4   t.amount,
5   t.declined,
6   c.company_name,
7   c.country
8 FROM
9   `sprint3_silver.transactions_clean` AS t
10 JOIN
11   `sprint3_silver.companies_clean` AS c
12   ON t.business_id = c.company_id
13 WHERE
14   DATE(t.timestamp) = '2022-03-12'
15   AND c.country = 'Germany';
```

At the bottom of the editor, a green checkmark icon is followed by the text: 'This query will process 6.68 MB when run.'

- Esta consulta tiene un coste de 6.68 MB

Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

Escenari:

Després del diagnòstic de l'exercici anterior, hem confirmat que filtrar la taula de transaccions per data o per empresa provoca un Full Table Scan (lectura completa de la taula), cosa que és ineficient. Per solucionar-ho a la capa de consum (Gold), materialitzarem una versió optimitzada de la taula de fets.

Problema del sandbox: com que utilitzem un entorn gratuït sense targeta de crèdit, BigQuery esborra automàticament les dades de les particions amb més de 60 dies d'antiguitat. Atès que les nostres dades històriques són antigues, si particionem directament la taula, aquesta acabarà quedant buida. Per evitar-ho, ho resoldrem en dos passos: primer, actualitzarem automàticament les dates de les dades i, després, aplicarem l'optimització física.

 Tasca:

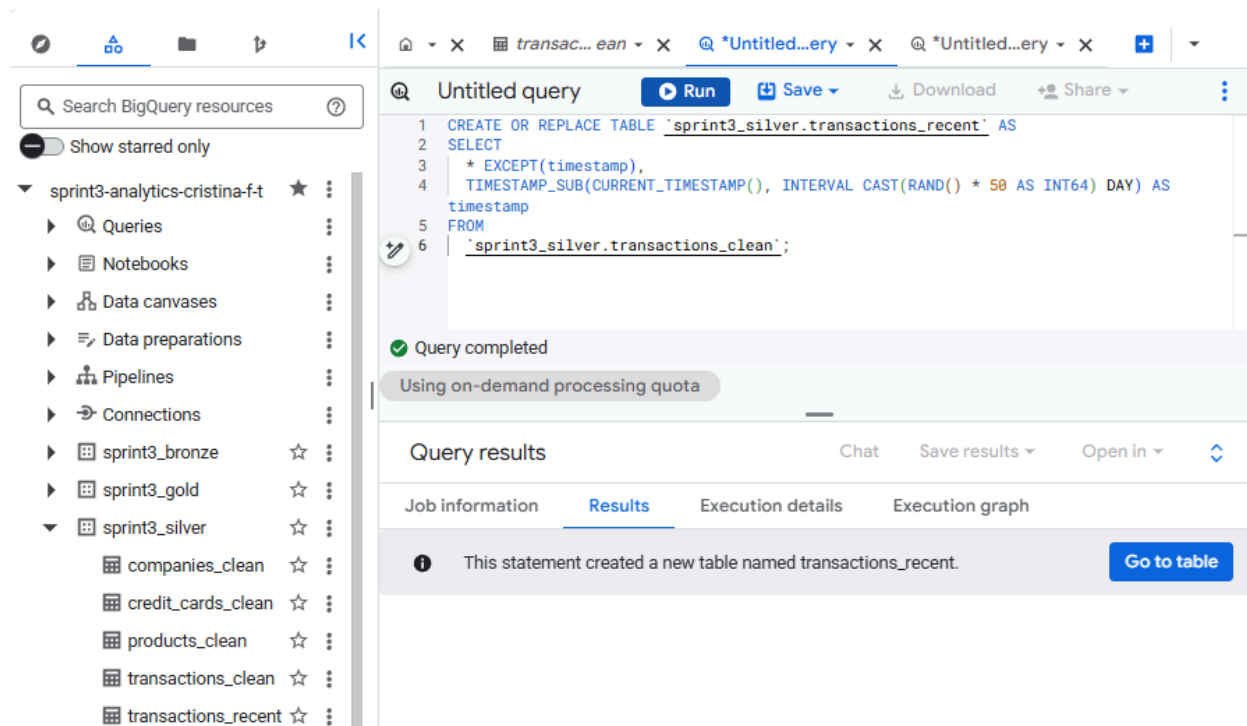
Pas 1: Generació de Dades Recents (Mocking Data)

Crea una taula intermèdia anomenada `sprint3_silver.transactions_recent` a partir de la taula `sprint3_silver.transactions_clean`. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

» Pista Tècnica: Per fer-ho sense escriure totes les columnes a mà, pots utilitzar `SELECT * EXCEPT(timestamp)`. Per calcular la data aleatòria, hauràs de restar una quantitat de dies a l'instant actual. Et seran molt útils les funcions `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, i la combinació `CAST(RAND() * 50 AS INT64)` per generar els dies de desfasament.

- [Generamos el código de creación de la tabla intermedia siguiendo las instrucciones de la pista técnica:](#)

```
CREATE OR REPLACE TABLE `sprint3_silver.transactions_recent` AS
SELECT
  * EXCEPT(timestamp),
  TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 50 AS INT64) DAY) AS
timestamp
FROM
  `sprint3_silver.transactions_clean`;
```



Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Ara, crea la teva taula definitiva `sprint3_gold.fact_transactions_optimized` a partir de les dades recents que acabes de generar a `transactions_recent`. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge:

- » Particionament (Partitioning): Divideix la taula per la data del camp `DATE(timestamp)`. Això permetrà que les consultes que filtrin per dia (`WHERE date = ...`) només llegeixin la partició necessària.
- » Clustering: Ordena les dades dins de cada partició per `business_id`. Això accelerarà dràsticament els filtres per companyia i els encreuaments (JOINS) amb la dimensió d'empreses.

👉 Entrega:

- » El codi SQL utilitzat per al Pas 1.
- » El codi DDL (`CREATE OR REPLACE TABLE...`) utilitzat al Pas 2.
- » Una captura de pantalla dels "Detalls" de la nova taula `fact_transactions_optimized` on es pugui comprovar que està particionada i agrupada en clústers correctament.

- Código de creación de la nueva tabla en el dataset Gold :

```
CREATE OR REPLACE TABLE `sprint3_gold.fact_transactions_optimized`
PARTITION BY DATE(timestamp)
CLUSTER BY business_id AS
SELECT
```

S4.01 Cristina Fornaguera Torner

```
transaction_id,  
card_id,  
business_id,  
user_id,  
amount,  
declined,  
lat,  
longitude,  
product_ids,  
timestamp  
FROM  
`sprint3_silver.transactions_recent`;
```

The screenshot displays the Google Cloud BigQuery interface. On the left, the resource explorer shows a project named 'sprint3-analytics-cristina-ft' with various resources like Queries, Notebooks, and Tables. The 'sprint3_silver' table is selected. The main panel shows a SQL query in the 'Untitled query' editor. The query is as follows:

```
1 CREATE OR REPLACE TABLE `sprint3_gold.fact_transactions_optimized`  
2 PARTITION BY DATE(timestamp)  
3 CLUSTER BY business_id AS  
4 SELECT  
5   transaction_id,  
6   card_id,  
7   business_id,  
8   user_id,  
9   amount,  
10  declined,  
11  lat,  
12  longitude,  
13  product_ids,  
14  timestamp  
15 FROM  
16 `sprint3_silver.transactions_recent`;
```

Below the query editor, a status bar indicates 'Query completed' and 'Using on-demand processing quota'. The 'Query results' section is visible, showing tabs for 'Job information', 'Results', 'Execution details', and 'Execution graph'. The 'Execution details' tab is active, displaying the following performance metrics:

Elapsed time	Slot time consumed	Bytes shuffled	Bytes spilled to disk
2.77 sec	33.14 sec	29.11 MB	0 B

Search BigQuery resources

fact_transactions_optimized

This is a partitioned table. [Learn more](#)

Schema Details Preview Table Explorer Preview Insights Lineage Data Profile

Table Type: Partitioned

Partitioned by: DAY

Partitioned on field: timestamp

Partition expiration: 60 days

Partition filter: Not required

Clustered by: business_id

Storage info

fact_transactions_optimized

Schema Details Preview Table Explorer Preview Insights Lineage Data Profile Data Quality

Row	transaction_id	card_id	business_id	user_id	amount	declined	lat	longitude	product_ids	timestamp
1	7C1E9739-7136-4031-A509-56...	CcS-8666	b-2222	4085	6.9	0	51.0679633...	10.1600270...	95	2026-05-06 09:48:44.632
2	C79D7026-929A-4289-8258-E8...	CcS-8248	b-2222	3667	306.76	0	52.1526763...	18.9469498...	32	2026-05-06 09:48:44.632
									34	
3	09CFDE7F-9B51-4E17-90CE-62...	CcU-3582	b-2222	183	161.11	0	55.1170315...	-3.46960221...	1	2026-05-06 09:48:44.632
4	DA063669-6C81-4A32-9CE2-CF...	CcS-7963	b-2222	3382	181.6	0	60.0697358...	18.6271695...	12	2026-05-06 09:48:44.632
5	74E490AD-203A-4390-A883-D5...	CcS-4974	b-2222	393	192.48	0	51.5125753...	18.8520989...	36	2026-05-06 09:48:44.632
									87	
6	0423B530-D74D-48A1-BB01-52...	CcS-5446	b-2222	865	352.92	0	45.7600479...	4.83463792...	1	2026-05-06 09:48:44.632
									34	
									7	
7	E31819E8-EF16-42E2-B830-AE3...	CcS-4899	b-2222	318	615.51	0	42.2419122...	12.5799501...	45	2026-05-06 09:48:44.632
									10	
									18	

Exercici 3: La Prova del Cotó (Benchmark)

Escenari:

Escenari: Ja has creat la taula `sprint3_gold.fact_transactions_optimized` amb les estratègies de Particionament i Clustering. Ara toca demostrar a l'equip la millora de rendiment i l'estalvi de costos que has aconseguit. Simularem una petició típica de negoci: "Volem veure totes les transaccions dels últims 30 dies".

🌱 Tasca:

L'objectiu és clar i directe: aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional.

Construeix una consulta SQL que seleccioni totes les columnes (SELECT *) i filtri les dades dels últims 30 dies.

⚠️ NO executis les consultes encara. En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els Bytes processats de manera gratuïta.

Fes la comparativa seguint aquests dos passos:

» Pas 1 (Taula no optimitzada):

Apunta la teva consulta a la taula `sprint3_silver.transactions_recent`. Observa el validador i fes una captura de pantalla dels bytes que processaria.

» Pas 2 (Taula optimitzada):

Sense canviar res de la lògica de la teva consulta, modifica només el FROM perquè ara apunti a la taula particionada `sprint3_gold.fact_transactions_optimized`. Observa com canvia el validador i fes la segona captura.

👉 Entrega:

- » El codi SQL base que has dissenyat.
- » Les dues captures de pantalla on es vegi la diferència de "Bytes processats" al validador.
- » Un breu text calculant quin és el % d'estalvi que has aconseguit (compara els bytes del Pas 1 respecte als del Pas 2).

- Construimos la consulta siguiendo las instrucciones (con SELECT *) sobre la tabla 'sprint3_silver.transactions_recent':

SELECT

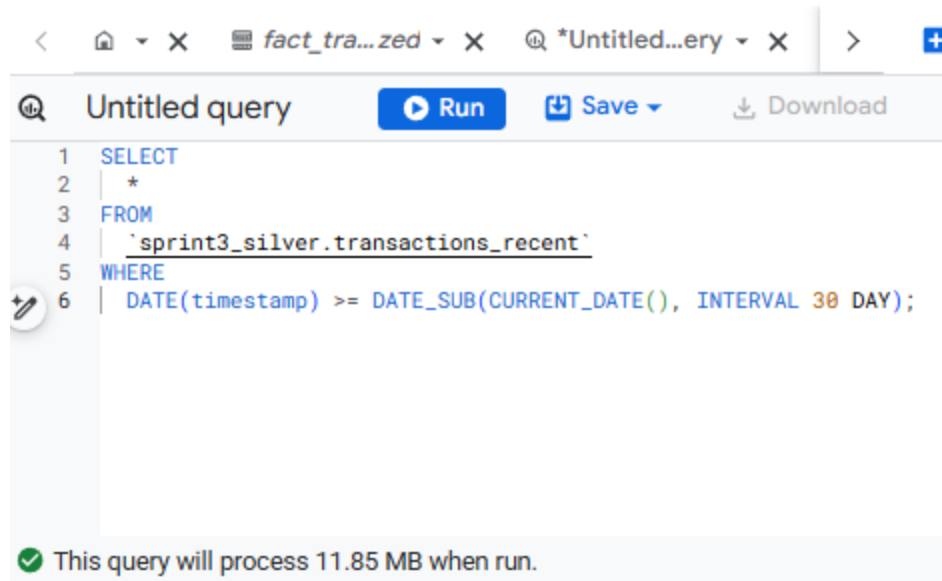
*

FROM

``sprint3_silver.transactions_recent``

WHERE

`DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);`



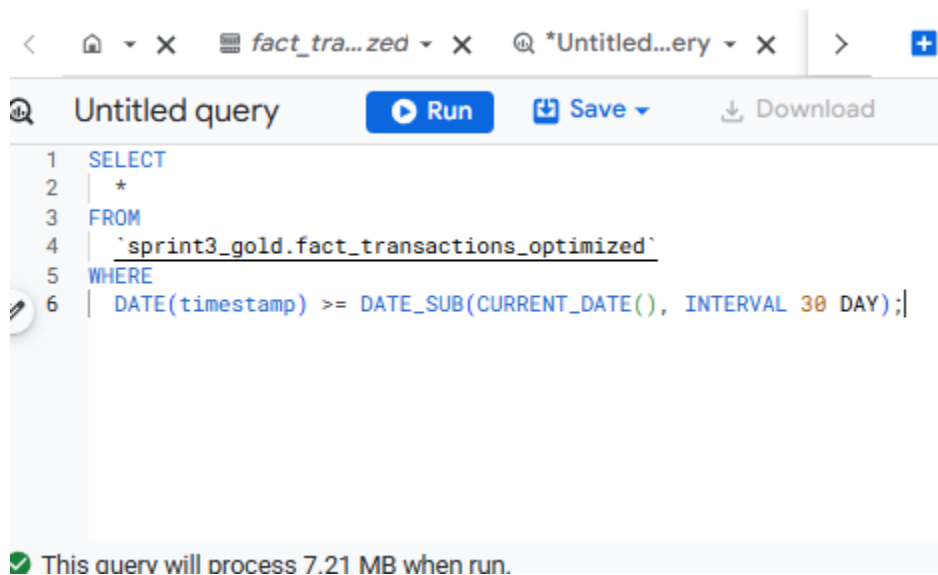
The screenshot shows a SQL query editor with a toolbar at the top containing icons for back, home, close, and tabs. The tabs are labeled 'fact_tra...zed', '*Untitled...ery', and a plus icon. Below the toolbar, the editor title is 'Untitled query' with buttons for 'Run', 'Save', and 'Download'. The SQL query is as follows:

```
1 SELECT
2   *
3 FROM
4   `sprint3_silver.transactions_recent`
5 WHERE
6   DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
```

Below the query, a green checkmark icon is followed by the text: 'This query will process 11.85 MB when run.'

- La aplicamos a la tabla 'sprint3_gold.fact_transactions_optimized'

```
SELECT
*
FROM
`sprint3_gold.fact_transactions_optimized`
WHERE
DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
```



The screenshot shows a SQL query editor with a toolbar at the top containing icons for back, home, close, and tabs. The tabs are labeled 'fact_tra...zed', '*Untitled...ery', and a plus icon. Below the toolbar, the editor title is 'Untitled query' with buttons for 'Run', 'Save', and 'Download'. The SQL query is as follows:

```
1 SELECT
2   *
3 FROM
4   `sprint3_gold.fact_transactions_optimized`
5 WHERE
6   DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
```

Below the query, a green checkmark icon is followed by the text: 'This query will process 7.21 MB when run.'

- Podemos observar que en el primer caso, el coste de la consulta sobre la tabla No Optimizada 'sprint3_silver.transactions_recent' es de **11.85 MB**

- En el segundo caso, de consulta a una tabla Optimizada 'sprint3_gold.fact_transactions_optimized' (con particionamiento y cluster) es de **7.21 MB**
- Hay un **39.16%** de reducción de coste computacional y de volumen de datos procesados en la consulta de la tabla optimizada 'sprint3_gold.fact_transactions_optimized'.
- Esto demuestra la eficiencia de la estrategia de particionamiento de tablas.

Exercici 4: Smart Caching (Vistes Materialitzades)

Escenari:

El Director General té un quadre de comandament que mostra les "Vendes Totals per Dia". Ell (i 50 managers més) refresquen aquest gràfic constantment.

Problema:

Si utilitzes una consulta normal o una Vista estàndard, cada vegada que algú refresca la pàgina, BigQuery recalcula la suma de milions de files. És lent i estàs pagant pel mateix càlcul una vegada i una altra.

Solució:

Utilitzar una Vista Materialitzada. Aquesta tecnologia guarda el resultat del càlcul a la memòria cau i només l'actualitza si entren dades noves (incremental).

🧩 Tasca:

» Crea una Vista Materialitzada anomenada sprint3_gold.mv_daily_sales que mostri les Vendes Totals per Dia.

👉 Entrega:

» Codi per Crear la Vista Materialitzada.

» Fes una consulta que mostri la vista creada i fes una captura de pantalla que mostri en el seu conjunt la vista materialitzada creada, amb la consulta executada i els bits processats.

- **Creemos el código de creación de la Vista Materializada**
'sprint3_gold.mv_daily_sales':

```
CREATE OR REPLACE MATERIALIZED VIEW `sprint3_gold.mv_daily_sales` AS
SELECT
  DATE(timestamp) AS sales_date,
  SUM(amount) AS total_sales
FROM
  `sprint3_gold.fact_transactions_optimized`
WHERE
```

```
declined = 0  
GROUP BY  
sales_date;
```

The screenshot shows a SQL query editor interface. At the top, there's a toolbar with a magnifying glass icon, the text "Untitled query", a "Run" button, a "Schedule" button, an "Open in" dropdown, and a "More" button with a gear icon. Below the toolbar, the SQL query is displayed in a monospaced font, with line numbers 1 through 10 on the left. The query is:
1 CREATE OR REPLACE MATERIALIZED VIEW `sprint3\_gold.mv\_daily\_sales` AS
2 SELECT
3 DATE(timestamp) AS sales_date,
4 SUM(amount) AS total_sales
5 FROM
6 `sprint3\_gold.fact\_transactions\_optimized`
7 WHERE
8 declined = 0
9 GROUP BY
10 sales_date;
Below the query, there's a status bar with a green checkmark icon and the text "Query completed".
Below the status bar, there's a section titled "Query results". Under this section, there are four tabs: "Job information", "Results", "Execution details", and "Execution graph". The "Results" tab is selected and highlighted. Below the tabs, there's a message box with an information icon and the text: "This statement created a new materialized view named mv_daily_sales."

```
1 CREATE OR REPLACE MATERIALIZED VIEW `sprint3_gold.mv_daily_sales` AS  
2 SELECT  
3 DATE(timestamp) AS sales_date,  
4 SUM(amount) AS total_sales  
5 FROM  
6 `sprint3_gold.fact_transactions_optimized`  
7 WHERE  
8 declined = 0  
9 GROUP BY  
10 sales_date;
```

Query completed

Query results

Job information Results Execution details Execution graph

This statement created a new materialized view named mv_daily_sales.

- Creamos el código de la consulta a la nueva vista :

```
SELECT  
sales_date,  
ROUND(total_sales,2) AS total_sales  
FROM  
`sprint3_gold.mv_daily_sales`;
```

Untitled query

Run Save

```
1 SELECT
2   sales_date,
3   ROUND(total_sales,2) AS total_sales
4 FROM
5   `sprint3_gold.mv_daily_sales`;
```

✓ This query will process 816 B when run.

Using on-demand processing quota

Query results

Job information

Results

Visualization

JSON

Row	sales_date	total_sales	
1	2026-05-23	509316.76	
2	2026-06-03	522181.0	
3	2026-05-27	504932.96	
4	2026-05-12	528908.03	
5	2026-05-03	506949.94	
6	2026-05-02	524701.65	
7	2026-04-26	516086.1	

Nivell 2: SQL Analític Avançat

Escenari:

La direcció comença a fer preguntes que no es resolen amb un simple GROUP BY. Necessitem anàlisi de tendències i comportament d'usuari.

Exercici 1: Perfilat de Clients VIP (Mètriques Agregades amb CTEs)

Escenari:

L'equip de Màrqueting vol analitzar el comportament dels nostres millors clients per dissenyar l'estratègia del pròxim any. Definim "VIP" com aquells amb una despesa acumulada superior a 500€. Necessiten un informe que, per a cada VIP, mostri el seu nom, contacte i el seu patró de compra: quantes vegades ha comprat, quant gasta de mitjana i quina va ser la seva compra rècord.

✖ Tasca:

» 1. Crea una CTE anomenada VIP_Stats que agrupi per usuari i calculi:

La Despesa Total (SUM).

La Quantitat de Transaccions (COUNT).

El Tiquet Mitjà (AVG), arrodonit a 2 decimals.

La Compra Màxima (MAX).

Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.

» Creua la CTE amb users_combined per obtenir les dades personals.

Requisits de Sortida:

Columnes: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.

Ordenat per total_gastat descendent.

- Creamos la consulta siguiendo las instrucciones sobre la tabla

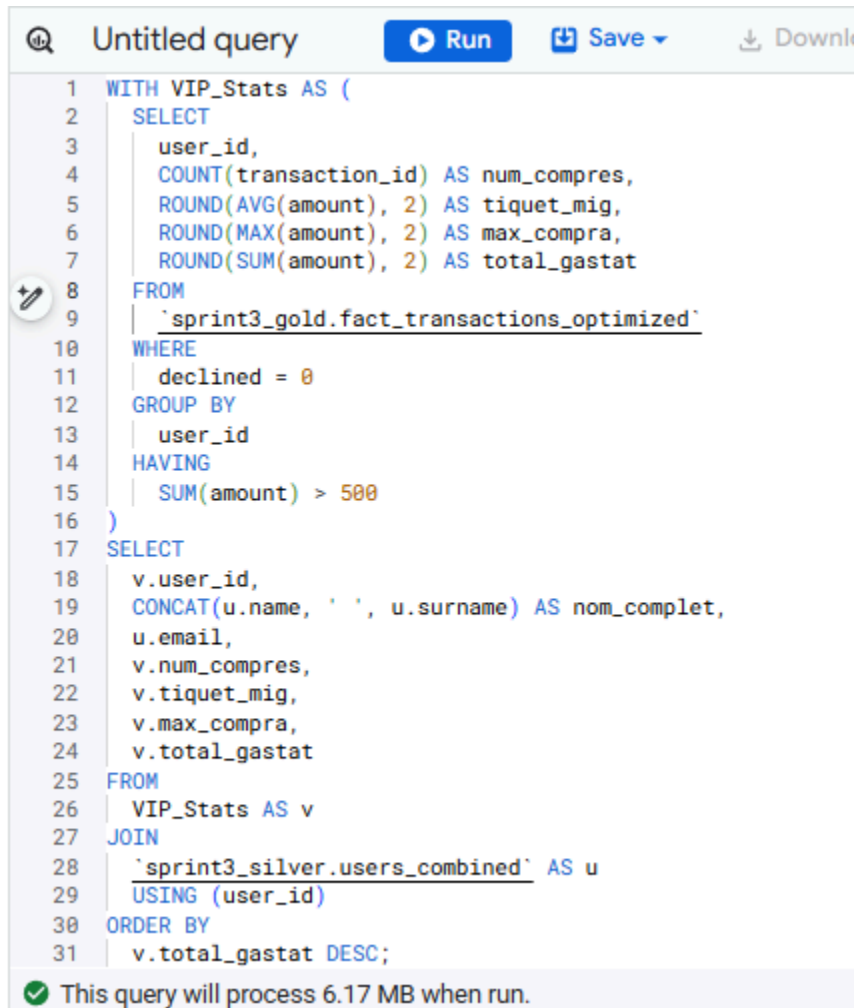
'sprint3_gold.fact_transactions_optimized':

```
WITH VIP_Stats AS (  
  SELECT  
    user_id,  
    COUNT(transaction_id) AS num_compres,  
    ROUND(AVG(amount), 2) AS tiquet_mig,  
    ROUND(MAX(amount), 2) AS max_compra,  
    ROUND(SUM(amount), 2) AS total_gastat  
  FROM  
    `sprint3_gold.fact_transactions_optimized`  
  WHERE  
    declined = 0  
  GROUP BY  
    user_id  
  HAVING  
    SUM(amount) > 500  
)  
SELECT  
  v.user_id,  
  CONCAT(u.name, ' ', u.surname) AS nom_complet,  
  u.email,  
  v.num_compres,  
  v.tiquet_mig,  
  v.max_compra,  
  v.total_gastat  
FROM  
  VIP_Stats AS v
```

JOIN

``sprint3_silver.users_combined` AS u``USING (user_id)`

ORDER BY

`v.total_gastat DESC;`

```
1 WITH VIP_Stats AS (  
2   SELECT  
3     user_id,  
4     COUNT(transaction_id) AS num_compres,  
5     ROUND(AVG(amount), 2) AS tiquet_mig,  
6     ROUND(MAX(amount), 2) AS max_compra,  
7     ROUND(SUM(amount), 2) AS total_gastat  
8   FROM  
9     `sprint3_gold.fact_transactions_optimized`  
10  WHERE  
11    declined = 0  
12  GROUP BY  
13    user_id  
14  HAVING  
15    SUM(amount) > 500  
16 )  
17 SELECT  
18   v.user_id,  
19   CONCAT(u.name, ' ', u.surname) AS nom_complet,  
20   u.email,  
21   v.num_compres,  
22   v.tiquet_mig,  
23   v.max_compra,  
24   v.total_gastat  
25 FROM  
26   VIP_Stats AS v  
27 JOIN  
28   `sprint3_silver.users_combined` AS u  
29   USING (user_id)  
30 ORDER BY  
31   v.total_gastat DESC;
```

✓ This query will process 6.17 MB when run.

- Esta consulta tiene un coste de 6.17 MB.

Untitled query Run Share Schedule Open in More Save Download

```

1 WITH VIP_Stats AS (
2   SELECT
3     user_id,
4     COUNT(transaction_id) AS num_compres,
5     ROUND(AVG(amount), 2) AS tiquet_mig,
6     ROUND(MAX(amount), 2) AS max_compra,
7     ROUND(SUM(amount), 2) AS total_gastat
8   FROM
9     'sprint3_gold.fact_transactions_optimized'
10  WHERE
11    declined = 0
12  GROUP BY
13    user_id
14  HAVING
15    SUM(amount) > 500
16 )
17 SELECT

```

Query completed

Using on-demand processing quota

Query results Chat Save results Open in

Job information **Results** Visualization JSON Execution details Execution graph

Row	user_id	nom_complet	email	num_compres	tiquet_mig	max_compra	total_gastat
1	289	Dxwgl Hwcru	dxwgl.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39
3	455	Fwfil Phylgmnhd	fwfil.phylgmnhd@example.com	66	522.59	724.44	34491.27
4	417	Fqatil Rdwcjvoa	fqatil.rdwcjvoa@example.com	73	421.87	675.78	30796.3
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	78	304.0	604.9	23711.9

Results per page: 50 1 - 50 of 5000

Exercici 2: Anàlisi de Tendències (Window Functions sobre Vistes)

Escenari:

La Direcció Financera vol monitoritzar la "velocitat de vendes" diària. Necessiten un informe que compari el rendiment de cada dia contra el dia anterior per detectar caigudes brusques o pics de creixement (Day-over-Day Growth).

Estratègia d'Optimització:

Per a aquest càlcul NO has d'anar a la taula de fets original ni recalculer sumes. Com a bon Data Engineer, has d'utilitzar la Vista Materialitzada `sprint3_gold.mv_daily_sales` que vas crear a l'exercici anterior. Això garanteix que l'anàlisi de tendències sigui instantània i no consumeixi recursos de còmput innecessaris.

Tasca:

Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

- » 1.Data: La data de venda.
- » 2.Vendes_Avui: El total venut aquell dia.
- » 3.Vendes_Ahir: El total venut el dia anterior (usant funcions de finestra).
- » 4.Diff_Percentual: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.



Pista Tècnica:

Utilitza la funció LAG(camp) OVER (ORDER BY data) per llegir el valor de la fila anterior sense necessitat de fer JOINS complexos.

- Creamos la consulta siguiendo las instrucciones sobre la vista materializada
`'sprint3_gold.mv_daily_sales'`

SELECT

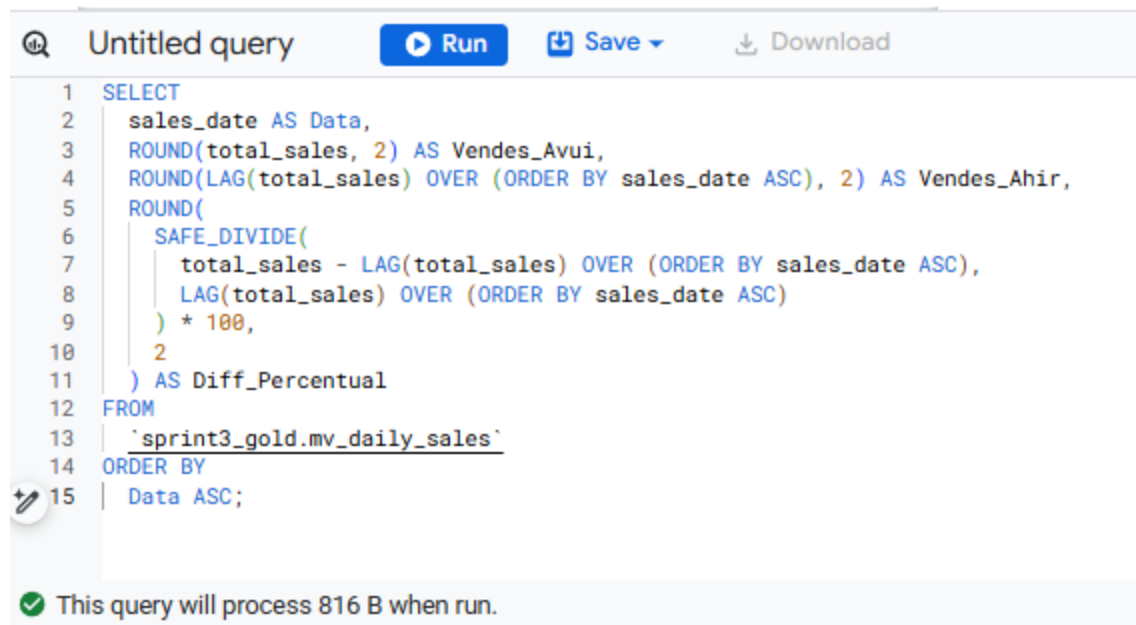
```
sales_date AS Data,  
ROUND(total_sales, 2) AS Vendes_Avui,  
ROUND(LAG(total_sales) OVER (ORDER BY sales_date ASC), 2) AS Vendes_Ahir,  
ROUND(  
    SAFE_DIVIDE(  
        total_sales - LAG(total_sales) OVER (ORDER BY sales_date ASC),  
        LAG(total_sales) OVER (ORDER BY sales_date ASC)  
    ) * 100,  
    2  
) AS Diff_Percentual
```

FROM

```
`sprint3_gold.mv_daily_sales`
```

ORDER BY

```
Data ASC;
```



The screenshot shows a SQL query editor with a toolbar at the top containing a search icon, the text "Untitled query", a "Run" button, a "Save" button with a dropdown arrow, and a "Download" button. The query is written in a monospaced font and is as follows:

```
1 SELECT
2   sales_date AS Data,
3   ROUND(total_sales, 2) AS Vendes_Avui,
4   ROUND(LAG(total_sales) OVER (ORDER BY sales_date ASC), 2) AS Vendes_Ahir,
5   ROUND(
6     SAFE_DIVIDE(
7       total_sales - LAG(total_sales) OVER (ORDER BY sales_date ASC),
8       LAG(total_sales) OVER (ORDER BY sales_date ASC)
9     ) * 100,
10    2
11  ) AS Diff_Percentual
12 FROM
13   `sprint3_gold.mv_daily_sales`
14 ORDER BY
15   Data ASC;
```

Below the query editor, a status bar displays a green checkmark icon followed by the text: "This query will process 816 B when run."

- Esta consulta tiene un coste de 816 B.

Untitled query Run Share Schedule Open in Mc

```

1 SELECT
2   sales_date AS Data,
3   ROUND(total_sales, 2) AS Vendes_Avui,
4   ROUND(LAG(total_sales) OVER (ORDER BY sales_date ASC), 2) AS Vendes_Ahir,
5   ROUND(
6     SAFE_DIVIDE(
7       total_sales - LAG(total_sales) OVER (ORDER BY sales_date ASC),
8       LAG(total_sales) OVER (ORDER BY sales_date ASC)
9     ) * 100,
10    2
11  ) AS Diff_Percentual
12 FROM
13   `sprint3_gold.mv_daily_sales`
14 ORDER BY
15   Data ASC;

```

✓ Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execution details	Executic
Row	Data	Vendes_Avui	Vendes_Ahir	Diff_Percentual	
1	2026-04-20	270179.44	null	null	
2	2026-04-21	516168.59	270179.44	91.05	
3	2026-04-22	533204.35	516168.59	3.3	
4	2026-04-23	519804.4	533204.35	-2.51	
5	2026-04-24	507064.38	519804.4	-2.45	
6	2026-04-25	519290.97	507064.38	2.41	
7	2026-04-26	516086.1	519290.97	-0.62	
8	2026-04-27	500831.97	516086.1	-2.96	

Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

Escenari:

El Director Financer (CFO) necessita visualitzar la corba de creixement anual. No li interessa tant el que es va vendre avui, sinó quant portem facturat en total des de l'inici de l'any fins avui (YTD - Year To Date).

✂ Tasca:

Utilitzant la vista materialitzada mv_daily_sales (per no recalculer agregacions base), genera un informe amb tres columnes:

» 1.Data.

» 2.Vendes del Dia: Arrodonit a 2 decimals.

» 3.Vendes Acumulades YTD: Una suma progressiva de les vendes que ha de reiniciar-se cada 1 de Gener. El resultat final ha d'estar arrodonit a 2 decimals.

💡 Pista Tècnica:

Per aconseguir el reinici anual, necessites dividir la finestra usant PARTITION BY EXTRACT(YEAR FROM data). L'estructura completa seria: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

- Crearemos la consulta siguiendo las instrucciones sobre la tabla 'sprint3_gold.mv_daily_sales':

```
SELECT
  sales_date AS Data,
  ROUND(total_sales, 2) AS Vendes_del_Dia,
  ROUND(
    SUM(total_sales) OVER (
      PARTITION BY EXTRACT(YEAR FROM sales_date)
      ORDER BY sales_date ASC
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ),
    2
  ) AS Vendes_Acumulades_YTD
FROM
  `sprint3_gold.mv_daily_sales`
ORDER BY
  Data ASC;
```

Untitled query Run Share Schedule Open in

```

1 SELECT
2   sales_date AS Data,
3   ROUND(total_sales, 2) AS Vendes_del_Dia,
4   ROUND(
5     SUM(total_sales) OVER (
6       PARTITION BY EXTRACT(YEAR FROM sales_date)
7       ORDER BY sales_date ASC
8       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
9     ),
10    2
11  ) AS Vendes_Acumulades_YTD
12 FROM
13   `sprint3_gold.mv_daily_sales`
14 ORDER BY
15   Data ASC;

```

✓ This query will process 816 B when run.

Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execution details
Row	Data	Vendes_del_Dia	Vendes_Acumulades_YTD	
1	2026-04-20	270179.44	270179.44	
2	2026-04-21	516168.59	786348.03	
3	2026-04-22	533204.35	1319552.38	
4	2026-04-23	519804.4	1839356.78	
5	2026-04-24	507064.38	2346421.16	
6	2026-04-25	519290.97	2865712.13	
7	2026-04-26	516086.1	3381798.23	
8	2026-04-27	500831.97	3882630.2	
9	2026-04-28	502084.64	4384714.84	

Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

Escenari:

L'equip de Màrqueting llança la campanya "A la tercera va la vençuda". Volen enviar un regal als usuaris que hagin completat la seva tercera transacció. No obstant això, el pressupost del regal no és fix: depèn de la "qualitat" del client en els seus inicis. Necessiten saber el Tiquet Mitjà d'aquestes 3 primeres compres per segmentar si envien un detall bàsic o un regal premium.

 Tasca:

Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

- » 1.Dades d'usuari (user_id, nom_complet, email).
- » 2.La data i l'import exacte de la 3a compra.
- » 3.Mitjana de les 3 primeres: La mitjana de despesa de les seves transaccions 1, 2 i 3.

💡 Pista Tècnica:

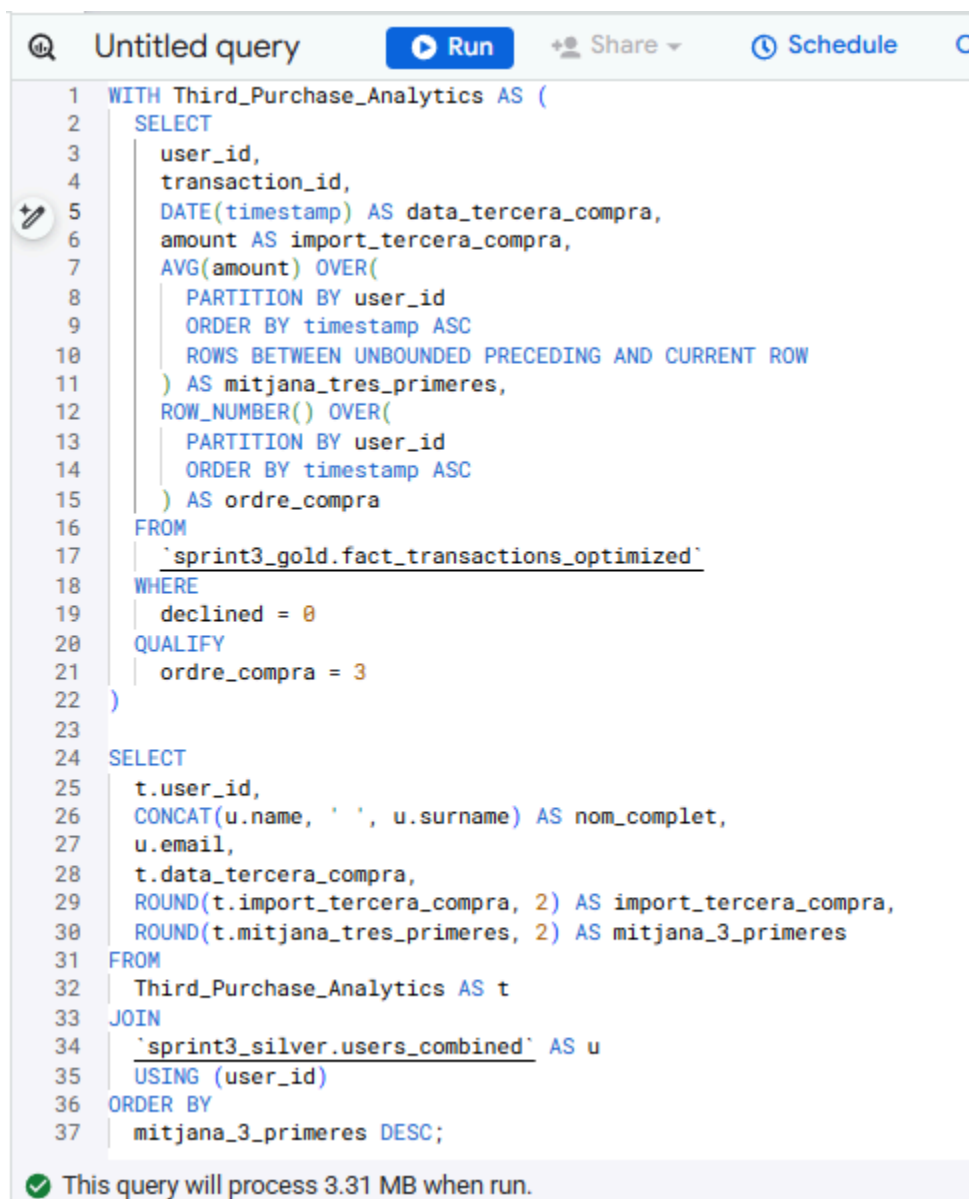
- » Has d'utilitzar la funció ROW_NUMBER() per establir l'ordre cronològic de les compres.
- » Has d'utilitzar la clàusula QUALIFY per filtrar.
- » Repte d'Optimització: Busca l'estratègia de menor cost computacional. Tingues en compte que calcular mitjanes sobre tot l'historial històric és costós. Intenta filtrar les files necessàries abans de realitzar l'agregació final.

- Creamos la consulta sobre la tabla 'sprint3_gold.fact_transactions_optimized':

```
WITH Third_Purchase_Analytics AS (  
  SELECT  
    user_id,  
    transaction_id,  
    DATE(timestamp) AS data_tercera_compra,  
    amount AS import_tercera_compra,  
    AVG(amount) OVER(  
      PARTITION BY user_id  
      ORDER BY timestamp ASC  
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS mitjana_tres_primeres,  
    ROW_NUMBER() OVER(  
      PARTITION BY user_id  
      ORDER BY timestamp ASC  
    ) AS ordre_compra  
  FROM  
    `sprint3_gold.fact_transactions_optimized`  
  WHERE  
    declined = 0  
  QUALIFY  
    ordre_compra = 3  
)  
  
SELECT  
  t.user_id,  
  CONCAT(u.name, ' ', u.surname) AS nom_complet,  
  u.email,  
  t.data_tercera_compra,
```

S4.01 Cristina Fornaguera Torner

```
ROUND(t.import_tercera_compra, 2) AS import_tercera_compra,  
ROUND(t.mitjana_tres_primeres, 2) AS mitjana_3_primeres  
FROM  
  Third_Purchase_Analytics AS t  
JOIN  
  `sprint3_silver.users_combined` AS u  
  USING (user_id)  
ORDER BY  
  mitjana_3_primeres DESC;
```



The screenshot shows a SQL query editor interface with a toolbar at the top containing 'Run', 'Share', and 'Schedule' buttons. The query is written in a light blue monospace font on a white background. It defines a CTE named 'Third_Purchase_Analytics' which selects transaction data from 'sprint3_gold.fact_transactions_optimized', applying a moving average window function to calculate 'mitjana_tres_primeres' and a row number to calculate 'ordre_compra'. The main query then joins this CTE with 'sprint3_silver.users_combined' on 'user_id' and orders the results by 'mitjana_3_primeres' in descending order. A status bar at the bottom indicates the query will process 3.31 MB.

```
1 WITH Third_Purchase_Analytics AS (  
2   SELECT  
3     user_id,  
4     transaction_id,  
5     DATE(timestamp) AS data_tercera_compra,  
6     amount AS import_tercera_compra,  
7     AVG(amount) OVER(  
8       PARTITION BY user_id  
9       ORDER BY timestamp ASC  
10      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
11    ) AS mitjana_tres_primeres,  
12    ROW_NUMBER() OVER(  
13      PARTITION BY user_id  
14      ORDER BY timestamp ASC  
15    ) AS ordre_compra  
16  FROM  
17    `sprint3_gold.fact_transactions_optimized`  
18  WHERE  
19    declined = 0  
20  QUALIFY  
21    ordre_compra = 3  
22 )  
23  
24 SELECT  
25   t.user_id,  
26   CONCAT(u.name, ' ', u.surname) AS nom_complet,  
27   u.email,  
28   t.data_tercera_compra,  
29   ROUND(t.import_tercera_compra, 2) AS import_tercera_compra,  
30   ROUND(t.mitjana_tres_primeres, 2) AS mitjana_3_primeres  
31 FROM  
32   Third_Purchase_Analytics AS t  
33 JOIN  
34   `sprint3_silver.users_combined` AS u  
35   USING (user_id)  
36 ORDER BY  
37   mitjana_3_primeres DESC;
```

✓ This query will process 3.31 MB when run.

S4.01 Cristina Fornaguera Torner

Untitled query

Run

Share

Schedule

Open in

More

Save

Download

```
1 WITH Third_Purchase_Analytics AS (  
2   SELECT  
3     user_id,  
4     transaction_id,  
5     DATE(timestamp) AS data_tercera_compra,  
6     amount AS import_tercera_compra,  
7     AVG(amount) OVER(  
8       PARTITION BY user_id  
9       ORDER BY timestamp ASC  
10      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
11    ) AS mitjana_tres_primeres,  
12    ROW_NUMBER() OVER(  
13      PARTITION BY user_id  
14      ORDER BY timestamp ASC  
15    ) AS ordre_compra  
16  FROM  
17    'sprint3_gold.fact_transactions_optimized'  
18  WHERE  
19    declined = 0  
20  QUALIFY
```

✔ This query will process 3.31 MB when run.

Using on-demand processing quota

Query results

Chat

Save results

Oper

Job information

Results

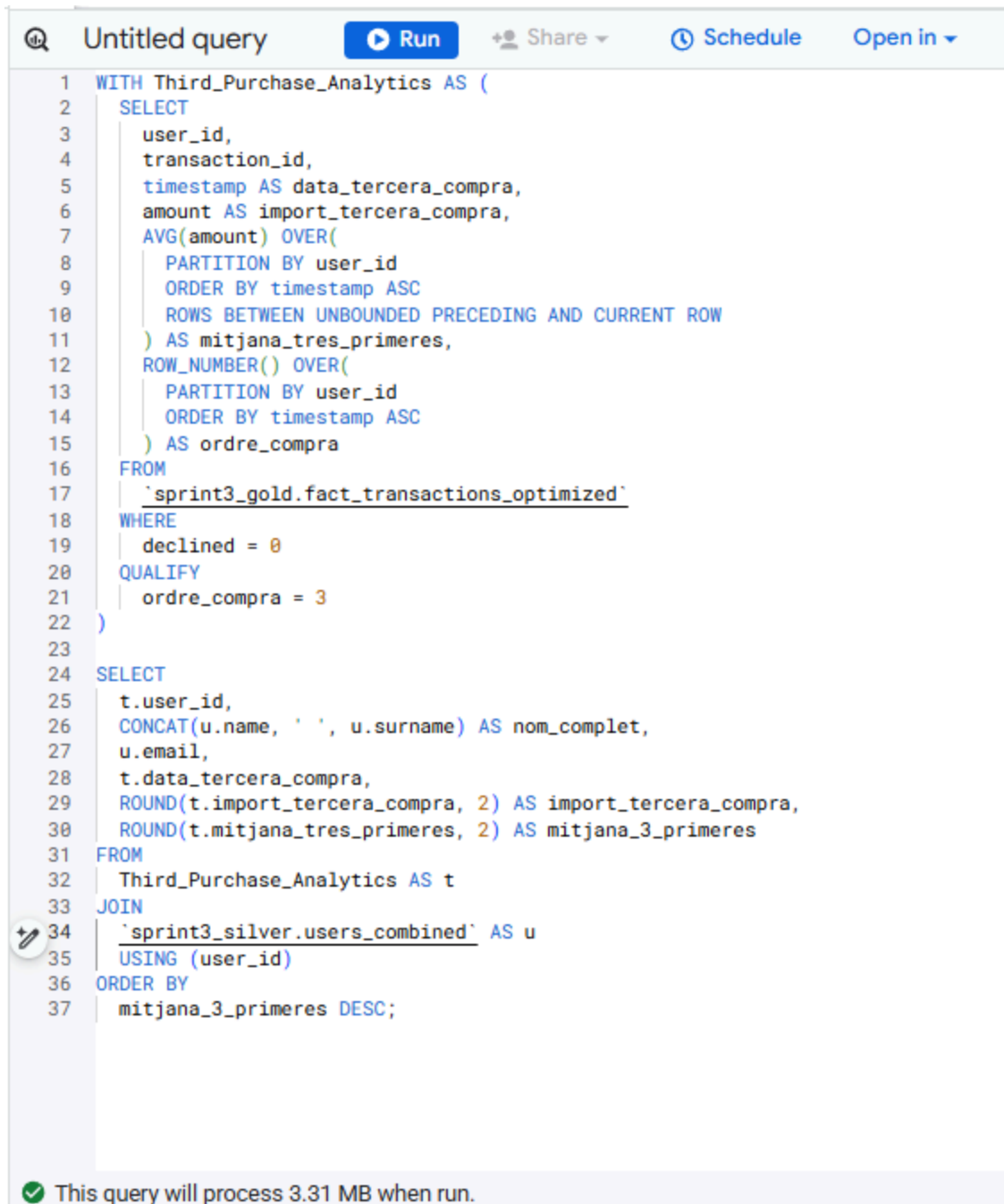
Visualization

JSON

Execution details

Execution graph

Row	user_id	nom_complet	email	data_tercera_compra	import_tercera_compra	mitjana_3_primeres
1	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	2026-04-22	630.52	630.52
2	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	2026-04-21	589.15	589.15
3	3048	Gqcfyy Mpifnltn	gqcfyy.mpifnltn@example.com	2026-05-08	601.9	572.35
4	4902	Sjzbez Gplbwibm	sjzbez.gplbwibm@example.com	2026-05-05	721.5	560.14
5	1757	Mxehow Eocgopkb	mxehow.eocgopkb@example.c...	2026-04-25	667.43	551.5
6	2194	Yjzhln Nyzdkkce	yjzhln.nyzdkkce@example.com	2026-04-26	511.53	540.1



```
1 WITH Third_Purchase_Analytics AS (  
2   SELECT  
3     user_id,  
4     transaction_id,  
5     timestamp AS data_tercera_compra,  
6     amount AS import_tercera_compra,  
7     AVG(amount) OVER(  
8       PARTITION BY user_id  
9       ORDER BY timestamp ASC  
10      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
11    ) AS mitjana_tres_primeres,  
12    ROW_NUMBER() OVER(  
13      PARTITION BY user_id  
14      ORDER BY timestamp ASC  
15    ) AS ordre_compra  
16  FROM  
17    `sprint3_gold.fact_transactions_optimized`  
18  WHERE  
19    declined = 0  
20  QUALIFY  
21    ordre_compra = 3  
22 )  
23  
24 SELECT  
25   t.user_id,  
26   CONCAT(u.name, ' ', u.surname) AS nom_complet,  
27   u.email,  
28   t.data_tercera_compra,  
29   ROUND(t.import_tercera_compra, 2) AS import_tercera_compra,  
30   ROUND(t.mitjana_tres_primeres, 2) AS mitjana_3_primeres  
31 FROM  
32   Third_Purchase_Analytics AS t  
33 JOIN  
34   `sprint3_silver.users_combined` AS u  
35   USING (user_id)  
36 ORDER BY  
37   mitjana_3_primeres DESC;
```

✓ This query will process 3.31 MB when run.

Nivell 3: Analytics Engineering (Arrays & Automatització)

Escenari:

Màrqueting necessita urgentment un Rànquing de Productes més Venuts.

Exercici 1: Desaniuament i Aplanament de Dades (Unnesting)

Escenari:

Actualment, a la taula de transaccions, els productes venuts estan "amagats" dins d'un Array (una llista d'IDs). L'equip de vendes vol analitzar quins productes específics es venen més, però no poden fer-ho si les dades estan comprimides en una llista. Necessiten una Taula Detallada (Flat Table) on cada fila representi un producte venut, encara que això signifiqui que l'ID de la transacció es repeteixi.

Tasca:

Crea la taula `dim_transactions_flat` desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

Passos Tècnics:

- » 1. Utilitza `CROSS JOIN UNNEST(product_ids)` per transformar l'Array en files individuals.
- » 2. Realitza un `JOIN` amb la taula `products` per obtenir el nom (`name`) i preu (`price`) de cada ítem.

 Nota: Ves amb compte amb els tipus de dades en fer el `JOIN` (Array de `INT64` vs `Product_ID STRING`).

- » 3. No agrupeu el resultat. Volem veure el desglossament línia per línia.

Resultat Esperat (Visualització):

La teva taula final ha de veure's "plana", on una transacció amb 3 productes ocupa 3 files:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portàtil Bàsic	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratolí	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclat	45.50

Entrega:

- » Una captura de la Vista Prèvia de la teva taula on es vegi el DDL executada correctament.
- » Una captura de pantalla on es vegi l'esquema de la nova taula.

- **Creemos el código de creación de la tabla `'sprint3_gold.dim_transactions_flat'` siguiendo las instrucciones:**

```
CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
SELECT
  t.transaction_id,
  t.timestamp,
  ROUND(t.amount, 2) AS total_ticket,
```


S4.01 Cristina Fornaguera Torner

```
p.product_id AS product_sku,  
p.name AS product_name,  
ROUND(p.price, 2) AS product_price  
FROM  
  `sprint3_gold.fact_transactions_optimized` AS t  
CROSS JOIN  
  UNNEST(t.product_ids) AS single_product_id  
INNER JOIN  
  `sprint3_silver.products_clean` AS p  
  ON SAFE_CAST(single_product_id AS INT64) = p.product_id  
WHERE  
  t.declined = 0;
```

Untitled query

RunSaveDownloadShare

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
2 SELECT
3   t.transaction_id,
4   t.timestamp,
5   ROUND(t.amount, 2) AS total_ticket,
6   p.product_id AS product_sku,
7   p.name AS product_name,
8   ROUND(p.price, 2) AS product_price
9 FROM
10  `sprint3_gold.fact_transactions_optimized` AS t
11 CROSS JOIN
12  UNNEST(t.product_ids) AS single_product_id
13 INNER JOIN
14  `sprint3_silver.products_clean` AS p
15  ON SAFE_CAST(single_product_id AS INT64) = p.product_id
16 WHERE
17  t.declined = 0;
```

Query completed

Using on-demand processing quota

Query results

Job information	Results	Execution details	Execution graph
Job ID	sprint3-analytics-cristina-f-t:EU.bquxjob_25191ba0_19eb0892a79 (S)		
User	Cristina.Fornaguera@gmail.com		
Location	EU		
Creation time	Jun 10, 2026, 9:57:20 AM UTC+2		
Start time	Jun 10, 2026, 9:57:20 AM UTC+2		
End time	Jun 10, 2026, 9:57:22 AM UTC+2		
Duration	2 sec		
Bytes processed	7.85 MB		
Bytes billed	20 MB		
Slot milliseconds	6310		

sprint3-analytics-cristina-f-t / Datasets / sprint3_gold / Tables / dim_transactions_flat

☆ dim_transacti... 🔍 Query 🗨 Chat Open in ▾ 👤 Share ▾ 📄 Co

Schema Details Preview Table Explorer **Preview** Insights Lineage [

Filter Enter property name or value

☐	Field name	Type	Mode	Description	Key	Coll
☐	transaction_id	STRING	NULLABLE	-	-	-
☐	timestamp	TIMESTAMP	NULLABLE	-	-	-
☐	total_ticket	FLOAT	NULLABLE	-	-	-
☐	product_sku	INTEGER	NULLABLE	-	-	-
☐	product_name	STRING	NULLABLE	-	-	-
☐	product_price	FLOAT	NULLABLE	-	-	-

Exercici 2: El Rànquing de Vendes (Agregació Simple)

Escenari:

Ara que has preparat la infraestructura i disposes de la taula dim_transactions_flat (on cada fila equival a un producte venut), generar informes analítics és trivial. Ja no necessites funcions complexes de desanidament per respondre preguntes bàsiques de negoci.

🌱 Tasca:

Genera el Top 5 de productes més venuts en la història de la companyia.

🔧 Mètode:

Consulta directament la teva nova taula desnormalitzada (dim_transactions_flat).

» En ser la taula plana, simplement necessites una agrupació estàndard (GROUP BY) pel nom del producte i un compte (COUNT) de les files.

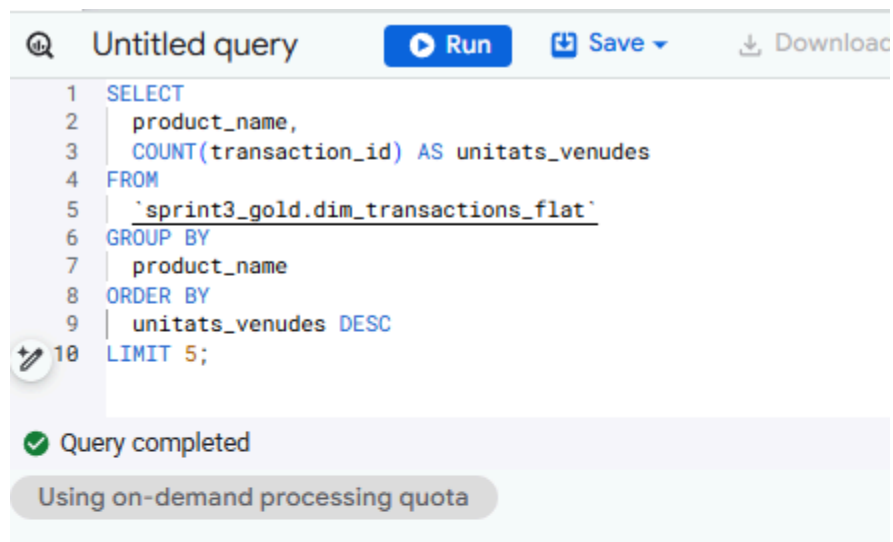
👉 Entrega:

» El codi SQL que genera el rànquing.

» Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

- Construimos la consulta a la tabla 'sprint3_gold.dim_transactions_flat' siguiendo las instrucciones:

```
SELECT
  product_name,
  COUNT(transaction_id) AS unitats_venudes
FROM
  `sprint3_gold.dim_transactions_flat`
GROUP BY
  product_name
ORDER BY
  unitats_venudes DESC
LIMIT 5;
```



The screenshot shows a SQL query editor interface. At the top, there's a search icon, the text "Untitled query", and buttons for "Run", "Save", and "Download". Below the header, the SQL query is displayed with line numbers 1 through 10. A green checkmark icon and the text "Query completed" are visible below the query. A grey button labeled "Using on-demand processing quota" is also present.

```
1 SELECT
2   product_name,
3   COUNT(transaction_id) AS unitats_venudes
4 FROM
5   `sprint3_gold.dim_transactions_flat`
6 GROUP BY
7   product_name
8 ORDER BY
9   unitats_venudes DESC
10 LIMIT 5;
```

Query completed

Using on-demand processing quota

Query results

Job information				Results	Visualization	JSON	Exec
Row	product_name	unitats_venudes					
1	Winterfell	10041					
2	the duel	10034					
3	dooku solo	10005					
4	Karstark Dorne	7666					
5	skywalker ewok	7582					

Exercici 3: Automatització del Pipeline i Visualització

Escenari:

L'informe estàtic de l'exercici anterior ha agradat molt, però el Director de Màrqueting necessita professionalitzar-lo per a la presa de decisions diària. Té dos requeriments urgents:

- » 1. Lògica Centralitzada: Volen veure el preu amb IVA (21%) desglossat. No obstant això, com que l'impost pot canviar per país o any, et prohibeixen posar el càlcul $\text{preu} * 1,21$ directament en el codi ("Hardcoded"). Has d'utilitzar una funció reutilitzable.
- » 2. Frescor de Dades: El tauler ha d'estar actualitzat automàticament cada matí a les 07:00 AM, abans de la reunió diària de vendes.

Tasca:

Has d'evolucionar la teva taula `dim_transactions_flat` per incloure el càlcul d'impostos i automatitzar la seva regeneració.

Passos Tècnics:

- » 1. User Defined Functions (UDF): Crea una funció SQL persistent anomenada `calculate_tax(amount)` que rebí un valor numèric i retorni el resultat aplicant el 21% d'impost.
- » 2. Integració i Orquestració:
Modifica el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció `calculate_tax` i generi una columna nova: `product_price_tax_inc`.
Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM.
- » 3. Visualització (BI): Connecta Looker Studio a la teva taula `dim_transactions_flat` i crea un Dashboard: "Monitor de Rendiment de Vendes"

Entrega:

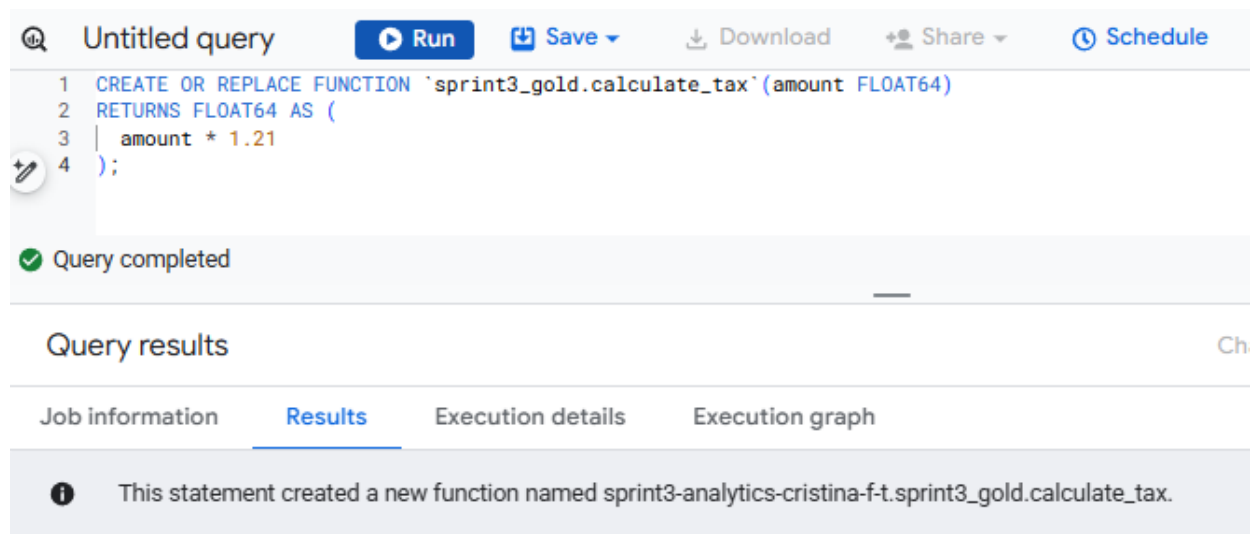
- » El codi SQL de la UDF.
- » El codi SQL actualitzat de la creació de la taula.
- » Una captura de pantalla demostrant que la Scheduled Query està programada.
- » Un enllaç del gràfic a Looker Studio.

Proposta de Dashboard:

"Monitor de Rendiment de Vendes"

- Generamos el código de la UDF :

```
CREATE OR REPLACE FUNCTION `sprint3_gold.calculate_tax`(amount FLOAT64)
RETURNS FLOAT64 AS (
  amount * 1.21
);
```



```

1 CREATE OR REPLACE FUNCTION `sprint3_gold.calculate_tax`(amount FLOAT64)
2 RETURNS FLOAT64 AS (
3   amount * 1.21
4 );

```

Query completed

Query results

Job information Results Execution details Execution graph

This statement created a new function named sprint3-analytics-cristina-f-t.sprint3_gold.calculate_tax.

- Creamos la tabla 'sprint3_gold.dim_transactions_flat' de nuevo incorporando el uso de la nueva función 'sprint3_gold.calculate_tax':

```

CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
SELECT
  t.transaction_id,
  t.timestamp,
  ROUND(t.amount, 2) AS total_ticket,
  p.product_id AS product_sku,
  p.name AS product_name,
  ROUND(p.price, 2) AS product_price,
  ROUND(`sprint3_gold.calculate_tax`(p.price), 2) AS product_price_tax_inc
FROM
  `sprint3_gold.fact_transactions_optimized` AS t
CROSS JOIN
  UNNEST(t.product_ids) AS single_product_id
JOIN
  `sprint3_silver.products_clean` AS p
  ON SAFE_CAST(single_product_id AS INT64) = p.product_id
WHERE
  t.declined = 0;

```

The screenshot shows a SQL query editor with a query titled "Untitled query". The query is as follows:

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
2 SELECT
3   t.transaction_id,
4   t.timestamp,
5   ROUND(t.amount, 2) AS total_ticket,
6   p.product_id AS product_sku,
7   p.name AS product_name,
8   ROUND(p.price, 2) AS product_price,
9   ROUND(`sprint3_gold.calculate_tax`(p.price), 2) AS product_price_tax_inc
10 FROM
11   `sprint3_gold.fact_transactions_optimized` AS t
12 CROSS JOIN
13   UNNEST(t.product_ids) AS single_product_id
14 JOIN
15   `sprint3_silver.products_clean` AS p
16   ON SAFE_CAST(single_product_id AS INT64) = p.product_id
17 WHERE
18   t.declined = 0;
```

On the right side, there is a settings panel with a green checkmark and the text "Settings valid." Below this, there is a section titled "Details and schedule" with a text input field for "Name for scheduled query *" containing the text "sprint3_gold_dim_transactions_flat_daily". Below this is a section titled "Schedule options" with a text input field for "Repeat frequency *" containing the text "Days" and a text input field for "At *" containing the text "07:00". At the bottom, there are two radio buttons: "Start now" (selected) and "Start at set time".

- No puedo programar la ejecución sin tener datos de pago en la cuenta, ejecuto el código anterior para actualizar la tabla:

The screenshot shows a SQL query editor with a query titled "Untitled query". The query is as follows:

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
2 SELECT
3   t.transaction_id,
4   t.timestamp,
5   ROUND(t.amount, 2) AS total_ticket,
6   p.product_id AS product_sku,
7   p.name AS product_name,
8   ROUND(p.price, 2) AS product_price,
9   ROUND(`sprint3_gold.calculate_tax`(p.price), 2) AS product_price_tax_inc
10 FROM
11   `sprint3_gold.fact_transactions_optimized` AS t
12 CROSS JOIN
13   UNNEST(t.product_ids) AS single_product_id
14 JOIN
15   `sprint3_silver.products_clean` AS p
16   ON SAFE_CAST(single_product_id AS INT64) = p.product_id
17 WHERE
18   t.declined = 0;
```

Below the query, there is a green checkmark and the text "Query completed". Below this, there is a grey button with the text "Using on-demand processing quota".

Below the query editor, there is a section titled "Query results". Below this, there is a tabbed interface with four tabs: "Job information", "Results" (selected), "Execution details", and "Execution graph". Below the tabs, there is a message with an information icon and the text "This statement replaced the table named dim_transactions_flat."

Configura una pàgina amb 3 seccions clau:

» 1. Els "Big Numbers" (KPIs Generals):

A la part superior, col·loca tres targetes de resultats (Scorecards) per donar context immediat.

KPI 1: Ingressos Totals (Amb IVA)

Mètrica: `SUM(product_price_tax_inc)`

Per què: Mostra el volum real de diners que entra.

Transaccions Úniques (Comandes)

Mètrica: `COUNT_DISTINCT(transaction_id)`

Transaccions Úniques (Comandes)

Camp Calculat: Crea un camp a Looker: `SUM(product_price_tax_inc) /`

`COUNT_DISTINCT(transaction_id)`

» 2. Evolució Temporal (Gràfic de Sèries Temporals)

Sota els KPIs, un gràfic de línia que ocupi tot l'ample.

Dimensió: timestamp (Desglossat per Data).

Mètrica: `SUM(product_price_tax_inc)`.

Línia de Referència (Opcional): Afegeix una mitjana constant o una línia de tendència.

Valor: Permet a Màrqueting veure si les vendes pugen o baixen dia a dia, o si hi va haver un pic per alguna campanya específica.

» El "Top Productes" Enrquit (Taula amb Mapa de Calor)

Oblida't del gràfic de barres simple. Utilitza una Taula amb Barres incrustades o Mapa de Calor.

Dimensió: `product_name`.

Mètrica 1: `COUNT(*)` → Renombra-ho a "Unitats Venudes".

Mètrica 2: `SUM(product_price_tax_inc)` → Renombra-ho a "Ingressos Totals".

Mètrica 3: `SUM(product_price_tax_inc) - SUM(product_unit_price)` → Renombra-ho a "IVA Recaptat" (Això demostra el valor de la teva UDF).

Estil: A la configuració d'estil de la taula, activa l'opció "Mostra número com a: Barra" o "Mapa de calor" per a la columna d'Ingressos.

- [Conectamos Data Studio \(antes Looker Studio\) a la tabla 'dim_transactions_flat' de nuestro BigQuery:](#)

S4.01 Cristina Fornaguera Torner

Untitled Report
File View Page Help

← Add data to report

Make your BigQuery reports load even faster with BigQuery BI Engine. [Learn More](#)

BigQuery
By Google
BigQuery is Google's fully managed, petabyte scale, low-cost analytics data warehouse. BigQuery charges for querying/processing of data. Those queries are charged to the credit card of the billing project.
[LEARN MORE](#) [REPORT AN ISSUE](#)

Project	Dataset	Table
Search Projects	Search Datasets	Search Tables
sprint3-analytics-cristina-f-t	sprint3_bronze	dim_transactions_flat
My First Project	sprint3_gold	fact_transactions_optimized
	sprint3_silver	mv_daily_sales
		product_sales_ranking
		v_marketing_kpis

- Creamos las tarjetas, el gráfico y la tabla siguiendo las instrucciones en Data Studio :

Monitor de Rendiment de Vendes i Facturació

Ingressos Totals (Amb IVA)

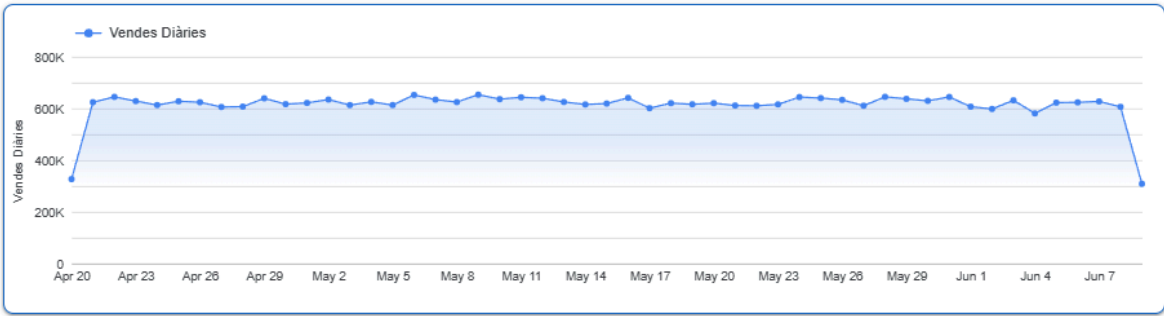
31,254,737.27 €

Transaccions Úniques

99,763

Tiquet Mitjà Global

313.29 €



Top Productes

Producte	Unitats Venudes ▾	Ingressos Totals	IVA Recaptat
Winterfell	10041	1,052,045.15 €	182,581.45 €
the duel	10034	1,270,666.43 €	220,521.82 €
dooku solo	10005	693,804.03 €	120,416.57 €
Karstark Dorne	7666	849,216.06 €	147,390.96 €
skywalker ewok	7582	1,533,910.33 €	266,213.20 €
Karstark warden	7539	1,243,414.99 €	215,797.48 €
Direwolf Stannis	7421	845,940.96 €	146,816.01 €
duel warden	5158	890,025.99 €	154,467.77 €

1 - 70 / 70 < >